

JEV vs OPENAI DECISIONS

Results + setup guide

Mark Kashef | October 2026

I tested OpenAI Decisions against TypeSafe Jev on 200 claims inside 40 fictional reports and 60 simulated incident investigations. Both scored 200/200 claims and 60/60 investigations on this frozen test set. The incident set repeats six fixed fault families ten times each; it does not contain 60 distinct faults.

The useful decision: test Jev when your input is text and unit cost matters. Test OpenAI Decisions when the decision needs a photo. Keep your own difficult examples in the evaluation before either one controls a real workflow.

This kit includes the complete 100 held-out runs and 857 request records, the exact prompts and inputs, score summaries, and a small Python comparison script. Historical results are from October 6, 2026. Setup documentation was checked October 8, 2026.

857 requests. Same scored answers. Different cost and input options.

Read the guide first. Use the included records to inspect the evidence. Run the starter script only when you are ready to use your own API accounts.

What the numbers say

Report checks: Jev 200/200, OpenAI 200/200. Median request time: 148.01 ms versus 141.82 ms. Estimated total input cost: \$0.004684554 versus \$0.006459900.

Incident investigations: Jev 60/60, OpenAI 60/60. Requests: 210 versus 247. Median request time: 163.52 ms versus 134.72 ms. Estimated total input cost: \$0.007069062 versus \$0.013004300.

Combined cost: Jev \$0.011753616; OpenAI \$0.019464200. Jev was 39.6% less expensive overall, 27.5% less on reports and 45.6% less on incidents. The Short says “about half”; the incident result is close to that, while the combined result is closer to 40%.

OpenAI was faster per call. Jev used fewer calls in the incident workflow, so its median summed API wait per investigation was lower: 468.81 ms versus 589.18 ms. Those are sums of API waits, not a production wall-clock SLA.

Costs use returned input-token counts multiplied by the dated public rates: Jev \$0.042 per million input tokens; OpenAI \$0.10 per million. These are estimates, not reconciled invoices. Both providers were called concurrently in pairs, one outstanding call per provider, with no automatic retries.

Set up both APIs

You need Python 3.10 or later, an OpenAI API account with Decisions access, and a TypeSafe API key. Billing and model access belong to each provider account. A ChatGPT subscription alone does not supply an API key.

Create keys in each provider dashboard. On macOS/Linux set `OPENAI_API_KEY` and `TYPESAFE_API_KEY` in your terminal session. On Windows PowerShell use `$env:OPENAI_API_KEY` and `$env:TYPESAFE_API_KEY`. Do not put keys in screenshots, prompts, source files or the downloaded kit.

Extract the kit, then run: `python3 compare.py --dry-run`. This validates the request shapes locally and makes no network requests. Then run: `python3 compare.py --live`. That makes one paid request to each provider using the same fictional claim and the same three answer choices.

Expected answer for the included example: contradicted. The evidence says revenue was \$80,000; the claim says \$100,000. The script rejects unexpected answer values and records choice, latency and usage. It does not retry failed or uncertain requests automatically.

To use your own case, copy `example-case.json` and edit evidence, claim and expected. Run: `python3 compare.py --live --case your-case.json --out your-results.json`. Use fictional or approved non-sensitive data. Review data-processing terms before sending customer material to either provider.

The same decision, two request shapes

TypeSafe endpoint: POST <https://api.typesafe.ai/v1/systemone>. Model: jev-1.13.0. Send state as the evidence plus claim, then questions.result with type “choice”, instructions and criteria mapping each allowed value to its definition.

OpenAI endpoint: POST <https://api.openai.com/v1/decisions>. Model: gpt-6-luna. Send input as the evidence plus claim, then questions as an array containing type “choice”, name “result”, instructions and choices, each with value and description.

The compare.py file contains the complete requests. Keep the allowed values identical: supported, contradicted, insufficient. A missing fact must be allowed to return insufficient. Otherwise the model is forced to guess.

Read Jev answers.result.choice and OpenAI answers[0].choice. Handle refusal, timeout, HTTP errors and missing or invalid choices as failures requiring review. A confidence score is not a measured probability of being correct on your data.

Photos and workflow boundaries

Jev in this comparison accepts text. For OpenAI Decisions, the checked guide supports inline images: input is a user message whose content contains input_text plus input_image. The image_url is a data URL, for example data:image/png;base64,... . Do not assume an external URL or uploaded file ID is accepted by this endpoint.

Use the image-request.json template as the request structure, replacing the placeholder with an actual base64 image. The kit does not run image requests automatically. Choose visible_damage, no_visible_damage or cannot_assess; an obscured object should not be treated as undamaged.

The original image demonstration used generated examples to establish input capability. It did not establish real-world damage-detection accuracy and is excluded from the scored text comparison. OCR before Jev changes the pipeline; measure its cost, latency and errors separately.

A model choice is a proposed next step. Keep refunds, payments, deletion and other consequential actions behind deterministic rules and human approval. Bound the number of workflow steps; stop on repeated choices, refusal, missing evidence or exhausted budget.

Test it on your work

Write the allowed outcomes and the evidence needed for each. Include an abstain or review outcome. Label a held-out set before seeing either provider answer, and keep prompt-tuning examples separate.

Include ambiguous language, missing facts, conflicting records, long inputs and instructions embedded inside the data. Score false approvals separately from overall accuracy. The included easy synthetic set reached a ceiling; a perfect score here is not proof of equal performance elsewhere.

Compare quality first, then p50/p95 latency, cost per correctly completed workflow, refusal/error rate and the number of calls needed. Freeze model versions and prompts, record the date, and count failed requests. Re-run when a provider changes the model or your input distribution changes.

Open results/requests.csv for request-level records. heldout-runs.json contains every full held-out input, choice, probability, observation and original result field. benchmark-summary.json has group scores. EVALUATION-MANIFEST.json links the frozen run IDs; PROMPT-FREEZE.json records source hashes. Earlier development runs are deliberately excluded from the reported scores.

Sources and reproducibility

Original Short: https://www.youtube.com/shorts/tDZrq3mp_Lo

Long-form comparison: https://www.youtube.com/watch?v=uTU5lhgl_7Q

TypeSafe request reference: <https://docs.typesafe.ai/api.md>

TypeSafe models and rates: <https://docs.typesafe.ai/models.md>

OpenAI Decisions guide: <https://developers.openai.com/api/docs/guides/decisions>

OpenAI request reference:

<https://developers.openai.com/api/reference/resources/decisions/methods/create>

The historical receipts are included so you can inspect my result without buying API calls. The starter script is a new, smaller example; it does not claim to reproduce the entire original benchmark. It has been checked locally in dry-run and response-parser tests. No new live API run is represented as part of the original results.